Project - Low Level Design
on
Real Estate AI Platform

Course Name: Devops

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1.	HARSHIT MITTAL	EN22CS301401
2.	HARSHAL DESHMUKH	EN22CS301396
3.	HARSHIT CHOUDHARY	EN22CS301400
4.	SALONEE CHOUHAN	EN23CS3L1017
5.	HARSHIT SHARMA	EN22CS301403
6.	HARSHIT BHARDWAJ	EN22CS301399

Group Name: Group 11D4

Project Number: DO-47

Industry Mentor Name: Mr. Vaibhav Sir

University Mentor Name: Prof. Avnesh Kumar Joshi

Academic Year: 2025-26

1. Introduction

The Real Estate AI Platform is a comprehensive full-stack application designed to modernize property browsing and automate the handling of customer inquiries. By integrating an AI triage workflow powered by a multi-agent architecture and a vector store for property-aware retrieval, the system can intelligently classify user intent, extract structured details, and generate contextually accurate responses.

1.1. Scope of the document

This document provides a detailed low-level design of the platform, including:

- System and layered architecture
- Application module design
- Data and process flows
- Core component design
- Key design considerations and devops infrastructure
- API structure and catalogue

1.2. Intended Audience

This document is intended for:

- Frontend and Backend Developers
- DevOps Engineers and System Administrators
- System Architects
- Project Mentors and Stakeholders

1.3. System overview

The platform is a robust full-stack application consisting of:

- A **React 18** and **Vite** frontend interface for users and admins.
- A **FastAPI** backend server managing core business logic.
- An AI processing pipeline utilizing **CrewAI** and the **Gemini 2.5 Flash** model.
- A **Chroma** vector database paired with Hugging Face embeddings for context retrieval.
- A **PostgreSQL** relational database for persistent data storage (managed via SQLAlchemy and Alembic).
- A robust **DevOps** suite utilizing Docker, Kubernetes, Terraform, Ansible, and Jenkins.

2. System Design

The system follows a highly decoupled, layered architecture to ensure scalability and maintainability:

- **Presentation Layer:** React-based frontend handling public routing, user dashboards, and admin panels.
- **Application Layer:** FastAPI backend exposing RESTful endpoints.
- **AI & Retrieval Layer:** CrewAI multi-agent pipeline working alongside Chroma vector search for Retrieval-Augmented Generation (RAG).
- **Data Layer:** PostgreSQL database for structured data (users, properties, records) and local file storage for uploads.

2.1. Application Design

The application is modularized into the following key domains:

Authentication Module

Handles secure user registration, login, and token validation using JWT auth.

Property Management Module

Allows public users to browse listings and view property details, while restricting property creation and deletion to authenticated administrators.

AI Triage & Concierge Module

Processes inquiries through a multi-step pipeline:

- Retrieves relevant property context using vector search.
- Classifies inquiry urgency and user intent via CrewAI and Gemini.
- Extracts structured details and drafts appropriate responses.

Admin Dashboard Module

Provides administrators with centralized access to view recent triage records, manage properties, and update inquiry statuses.

2.2. Process Flow

1. User registers or authenticates into the React frontend using JWT credentials.
2. User navigates to the AI assistant or property page and submits a real estate inquiry.
3. FastAPI backend receives the POST request.
4. The system queries the Chroma vector store to retrieve specific property context based on the user's prompt.

5. The TriageService invokes the CrewAI pipeline, injecting the retrieved context.
6. The Gemini model analyzes the data, classifies urgency, extracts details, and drafts a response.
7. The structured triage record is saved to the PostgreSQL database.
8. The drafted response is returned to the frontend for the user to view.
9. Administrators review the categorized inquiry via the admin dashboard and update its resolution status.

2.3. Information Flow

1. User input is captured via the React UI.
2. The payload is forwarded to the FastAPI backend.
3. The backend vectorizes the input and fetches context from Chroma DB.
4. Combined context and input are passed to the AI pipeline.
5. CrewAI and Gemini process the text and return structured JSON data.
6. The backend commits this structured data to PostgreSQL via SQLAlchemy.
7. The processed response is served back to the client UI.

2.4. Components Design

Core Services & Classes

- **AuthManager:** Manages user authentication, password hashing, and JWT token generation.
- **PropertyController:** Handles CRUD operations for real estate listings and interfaces with the database.
- **VectorRetriever:** Manages the integration with Chroma and Hugging Face embeddings for similarity searches.
- **CrewPipeline:** Executes the AI workflow, defining agent roles and task sequences.
- **TriageService:** Orchestrates the flow between the `VectorRetriever`, `CrewPipeline`, and database storage.

Data Models (SQLAlchemy)

- **User:** Stores credentials, contact information, and role definitions (user/admin).
- **Property:** Stores real estate listing metadata (ID, description, location, price).
- **TriageRecord:** Stores the parsed AI output, including user ID, property ID, urgency, intent, and current status.

2.5. Key Design Considerations

- **Retrieval-Augmented Generation (RAG):** The inclusion of a Chroma vector store ensures the AI provides hallucination-free, property-specific answers.
- **Infrastructure as Code (IaC):** Extensive use of Terraform, Ansible, and Docker ensures the environment is reproducible and easily deployable across different stages.

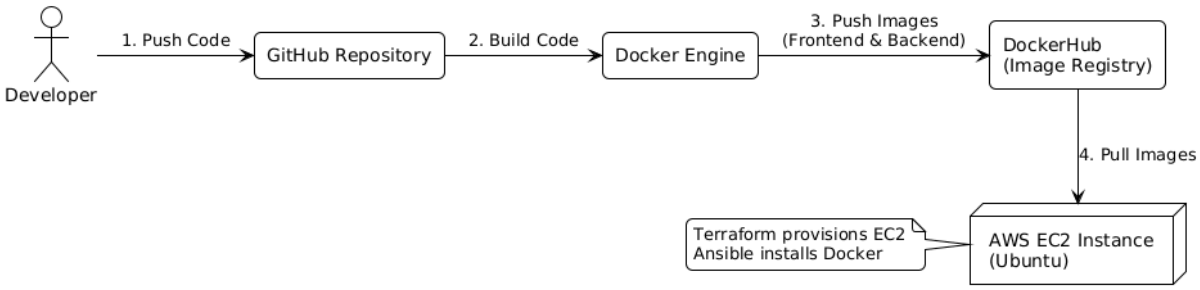
- **Separation of Concerns:** Distinct separation between the relational database (PostgreSQL) and the vector database (Chroma) optimizes querying performance for their respective data types.
- **Security:** Strict route protection on the frontend and backend ensures admin-only endpoints cannot be accessed by standard users.

2.6. API Catalogue

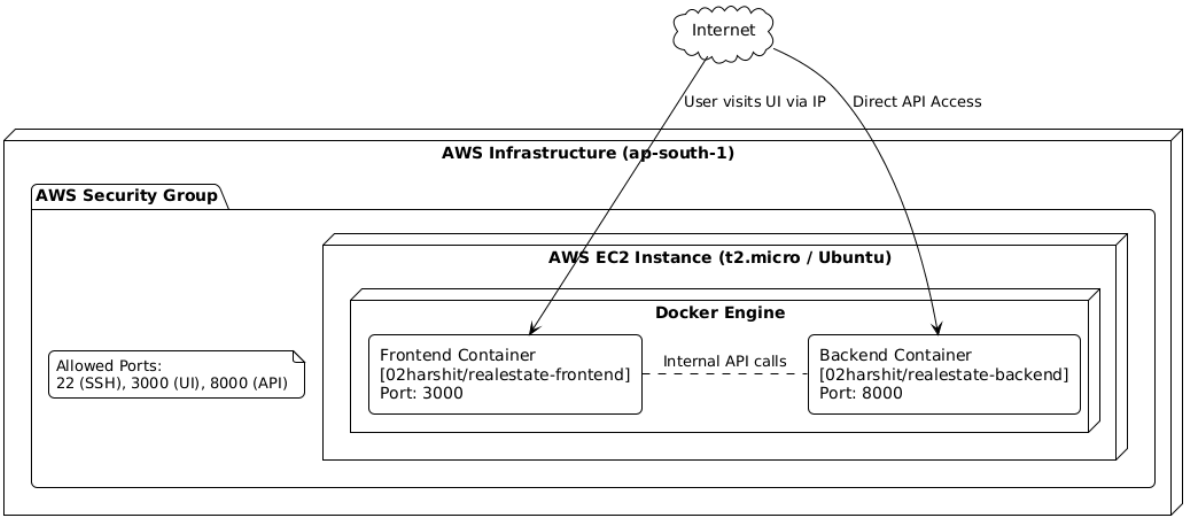
Endpoint	Method	Role	Description
`/register`	POST	Public	Registers a new user account.
`/login`	POST	Public	Authenticates a user and returns a JWT.
`/properties/`	GET	Public	Retrieves all property listings.
`/properties/{property_id}`	GET	Public	Retrieves details for a specific property.
`/triage`	POST	Authenticated	Submits an inquiry to the AI triage pipeline.
`/properties/`	POST	Admin	Creates a new

Endpoint	Method	Role	Description
			property listing.
/properties/{property_id}	DELETE	Admin	Removes an existing property listing.
/admin/records	GET	Admin	Retrieves a list of all stored triage records.
/admin/records/{record_id}/status	PATCH	Admin	Updates the status of a specific triage record.

DevOps Pipeline Diagram

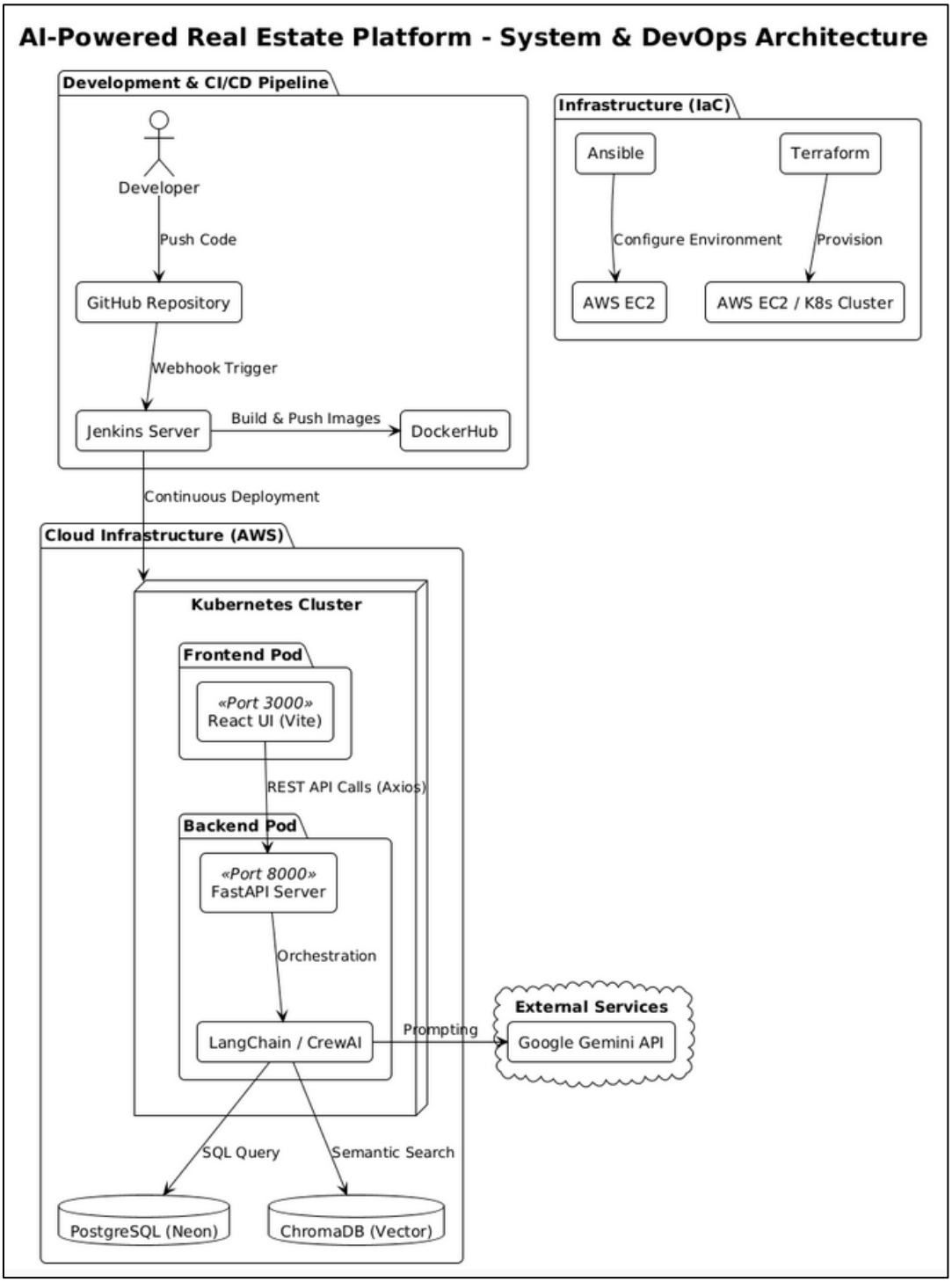


Deployment Architecture



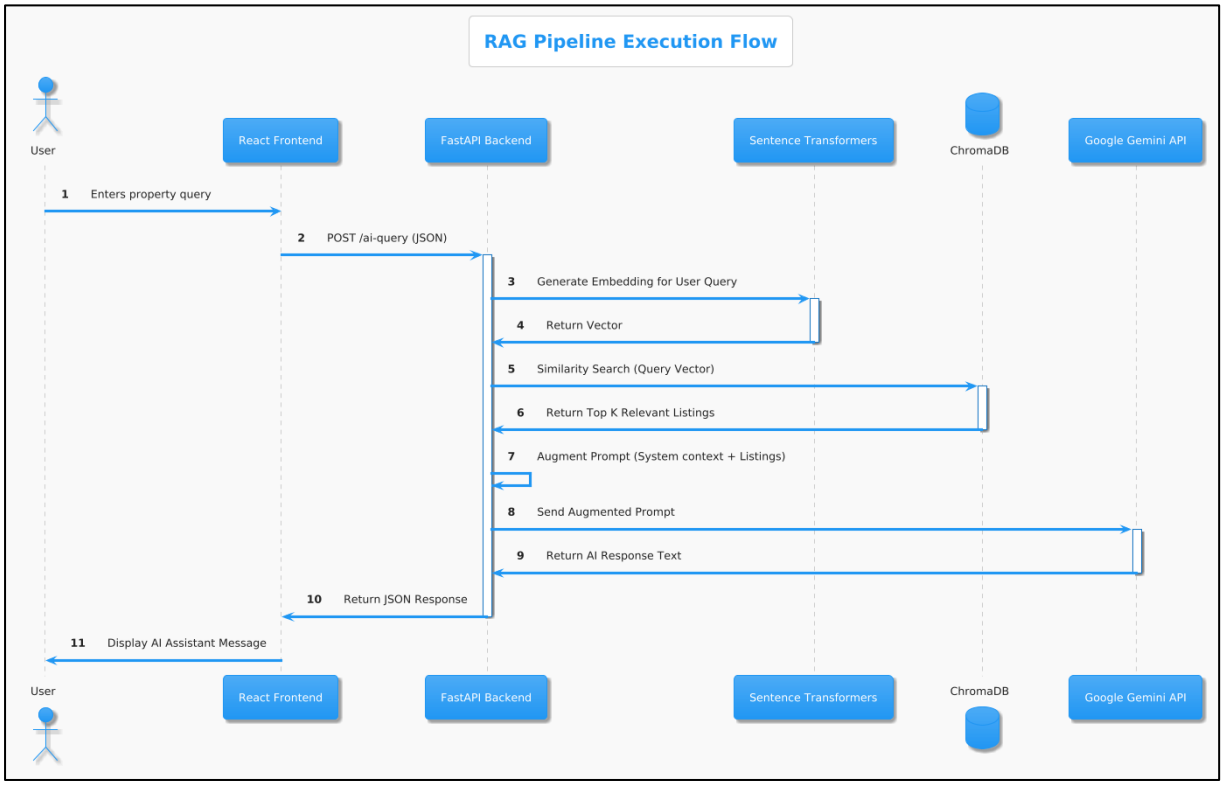
1. System & DevOps Architecture

This diagram visualizes the flow from the developer's machine through the CI/CD pipeline and into the AWS-hosted Kubernetes cluster.

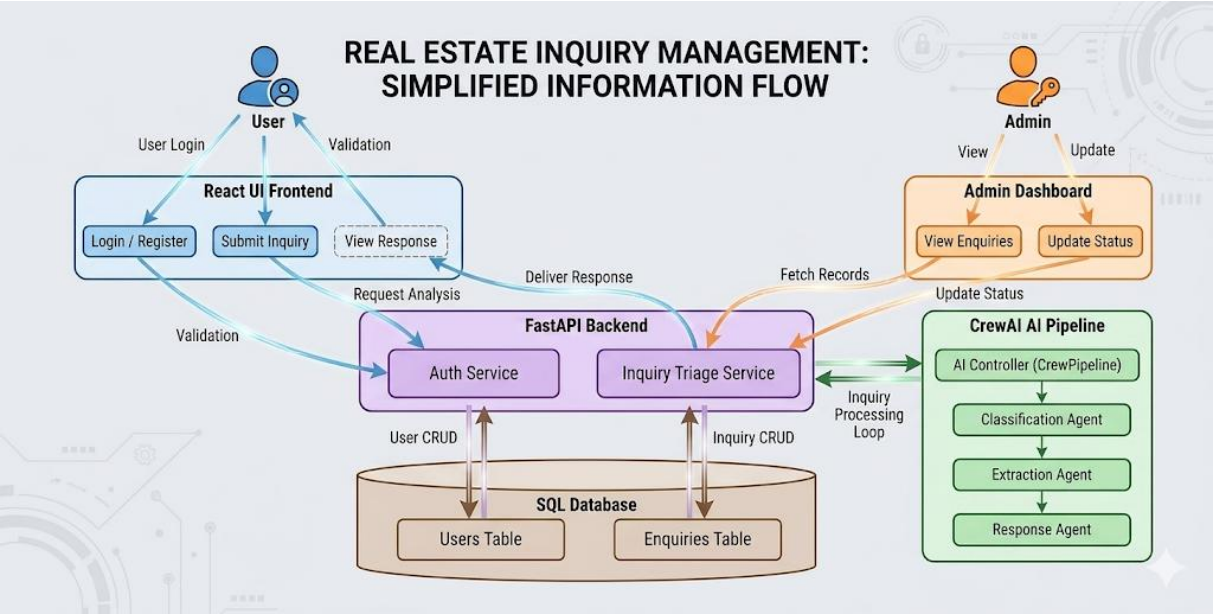


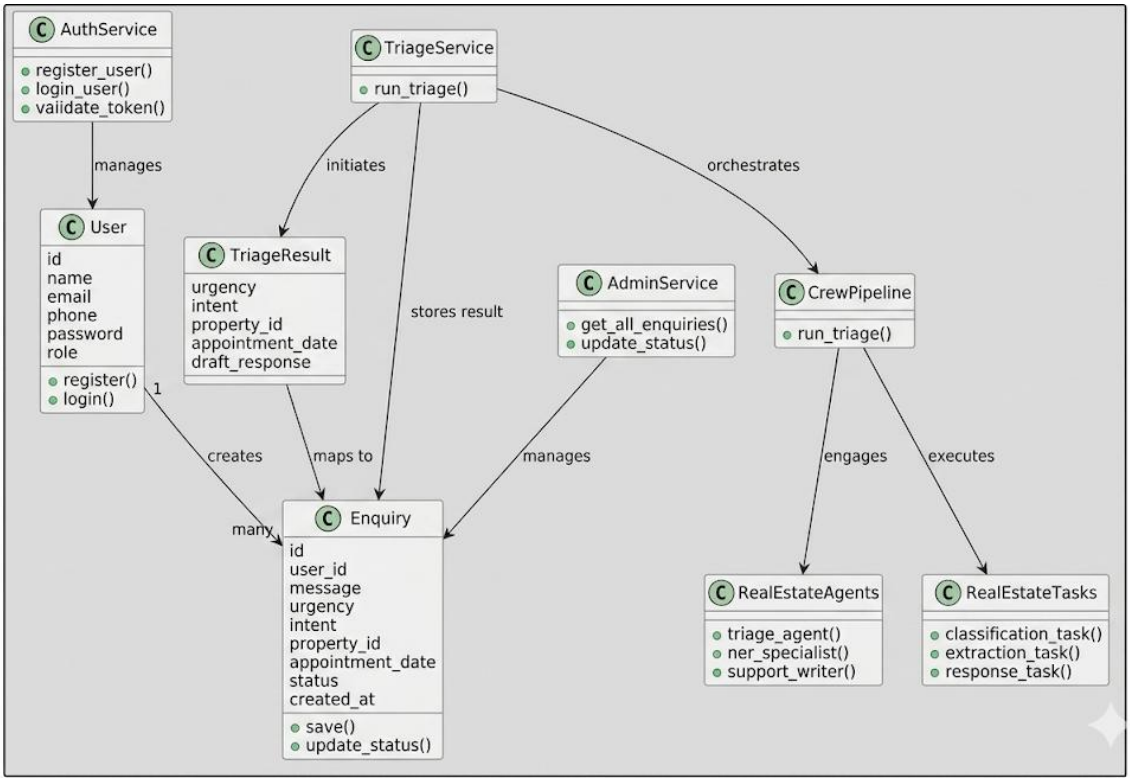
2. AI Processing Flow (RAG Sequence)

This diagram details Section 3 of your LLD, focusing on how a user query is transformed into an AI-generated response using the Retrieval Augmented Generation (RAG) pipeline.



Information Flow :





3. References

- React 18 & Vite Documentation
- FastAPI & Uvicorn Documentation
- PostgreSQL, SQLAlchemy, and Alembic Documentation
- CrewAI Framework Documentation
- Gemini API (2.5 Flash) Documentation
- Chroma Vector Database Documentation
- Docker & Kubernetes Official Documentation